

目录

1	企业级应用软件设计与开发 — 期末大作业报告	2
2	目录	2
3	一、选题背景与设计思想	3
3.1	1.1 问题定义	3
3.2	1.2 现有方案不足	3
3.3	1.3 项目价值	3
3.4	1.4 技术路线	3
4	二、Specs 规格文档	4
4.1	2.1 Product Spec 摘要	4
	4.1.1 核心功能	4
	4.1.2 用户故事	4
4.2	2.2 Architecture Spec 摘要	5
4.3	2.3 API Spec 摘要	5
4.4	2.4 Design Spec 摘要	5
4.5	2.5 Test Spec 摘要	5
5	三、系统架构与设计	6
5.1	3.1 整体架构	6
5.2	3.2 多智能体协作流程	7
5.3	3.3 中间件管道	7
5.4	3.4 数据流	8
6	四、关键实现与代码展示	9
6.1	4.1 Agent 核心循环	9
6.2	4.2 工具定义示例	10
6.3	4.3 MCP 气象 Agent	10
6.4	4.4 MCP 工具同步包装器	11
6.5	4.5 中间件实现	11
6.6	4.6 配置系统（SDD 驱动）	12
7	五、测试与评估	12
7.1	5.1 测试策略	12
7.2	5.2 RAGAS 评估结果	12
7.3	5.3 功能测试	13

1 企业级应用软件设计与开发 — 期末大作业报告	2
8 六、系统升级与扩展	13
8.1 6.1 可扩展架构	13
8.2 6.2 下一阶段计划	13
8.3 6.3 AI 能力演进路径	14
9 七、课程总结	14
9.1 7.1 个人收获	14
9.2 7.2 工程思维转变	14
9.3 7.3 对课程的建议	15
9.4 附录：技术栈一览	15
9.5 附录：核心技术要素覆盖	15

1 企业级应用软件设计与开发 — 期末大作业报告

课程代码：CS599 / 50120224001

项目名称：智扫通 — 多智能体协作的 Agentic RAG 智能客服系统

方向：方向一：Agentic AI 原生开发

学号：2025302932

姓名：李志鹏

专业：计算机技术

指导教师：戚欣

提交日期：2026 年 6 月 22 日

2 目录

1. 选题背景与设计思想
 2. Specs 规格文档
 3. 系统架构与设计
 4. 关键实现与代码展示
 5. 测试与评估
 6. 系统升级与扩展
 7. 课程总结
-

3 一、选题背景与设计思想

3.1 1.1 问题定义

智能家居市场快速增长，扫地/扫拖一体机器人已成为普及率最高的智能家居设备之一。然而，用户在实际使用中面临三大痛点：

- 选购迷茫：市面机型繁多（激光/视觉导航、单扫/扫拖一体、不同吸力规格），用户缺乏专业选购指导
- 故障无助：机器人出现“无法回充”“覆盖率低”“异响”等问题时，传统客服依赖人工排队，响应慢
- 维护知识碎片化：滤网更换周期、边刷清理频率、水箱保养等知识散布于说明书和论坛，用户难以及时获取

3.2 1.2 现有方案不足

现有方案	局限性
传统 FAQ 客服	关键词匹配，无法理解复杂语义，无上下文记忆
人工客服	成本高，响应慢，非工作时间不可用
通用 Chatbot	缺乏专业领域知识，无法调用外部 API（如天气），存在幻觉
单轮 RAG	无法多步骤推理，难以处理“查天气 → 给建议 → 生成报告”等复杂意图

3.3 1.3 项目价值

本系统利用 Agentic AI 技术，构建一个“听得懂、查得准、能推理”的扫地机器人智能客服。核心价值：

1. 自主推理：ReAct 循环下 Agent 自主决定调用哪些工具、以什么顺序调用
2. 多源融合：向量知识库（RAG）+ 外部 API（天气、定位）+ 结构化数据（CSV），统一编排
3. 智能切换：根据用户意图动态切换常规问答 / 报告生成两种模式
4. 流式交互：用户即时看到推理过程，体验接近真人对话

3.4 1.4 技术路线

层级	选型	理由
LLM	阿里云 Qwen3-Max (ChatTongyi)	中文能力强，DashScope API 稳定
Embedding	DashScope text-embedding-v4	与 LLM 同生态，1024 维向量
Agent 框架	LangGraph (create_agent)	课程推荐框架，原生支持 ReAct + middleware
向量数据库	Chroma	轻量本地持久化，无需额外部署
通信协议	MCP (Model Context Protocol)	连接独立气象 Agent，前沿技术加分
前端	Streamlit	快速构建 Chatbot UI，支持流式输出
评估	RAGAS 四大指标	客观量化 RAG 管道质量

4二、Specs 规格文档

完整 SDD 规格文档详见 docs/CS599_Specs.md。本章为摘要版，完整 Product Spec / Architecture Spec / API Spec / Design Spec / Test Spec 请参阅该文件。

4.12.1 Product Spec 摘要

4.1.1.1 核心功能

编号	功能	描述	优先级
F1	RAG 专业知识问答	向量库检索 + LLM 摘要，覆盖选购/故障/保养	P0
F2	天气环境分析	自动定位 + 天气查询，分析对机器人使用的影响	P1
F3	个人使用报告生成	查询月度使用记录，生成结构化分析报告	P1
F4	IP 自动定位	无需手动输入，自动识别用户城市	P2
F5	流式对话交互	实时流式输出，打字机效果	P2

4.1.1.2 用户故事

ID	作为	我想要	以便
US1	用户	询问故障怎么处理	快速解决使用问题
US2	用户	了解小户型适合什么机器人	做出正确购买决策

ID	作为	我想要	以便
US3	用户	知道今天天气是否适合拖地	合理安排清洁计划
US4	用户	查看本月使用报告	了解清洁效率和耗材情况

4.22.2 Architecture Spec 摘要

系统采用双智能体协作架构：

- 主控 Agent (ReAct): LangGraph 编排，5 个本地工具 + 3 个中间件 + Qwen3-Max
- 气象 Agent (MCP): 独立进程 FastMCP，暴露 get_weather / get_location 两个工具
- 通信方式: MCP 协议 stdio 管道，MultiServerMCPClient 自动发现 + 异步转同步包装

4.32.3 API Spec 摘要

7 类工具接口（5 个本地 + 2 个 MCP），完整入参/出参/异常/依赖定义见 docs/CS599_Specs.md 第三章。

4.42.4 Design Spec 摘要

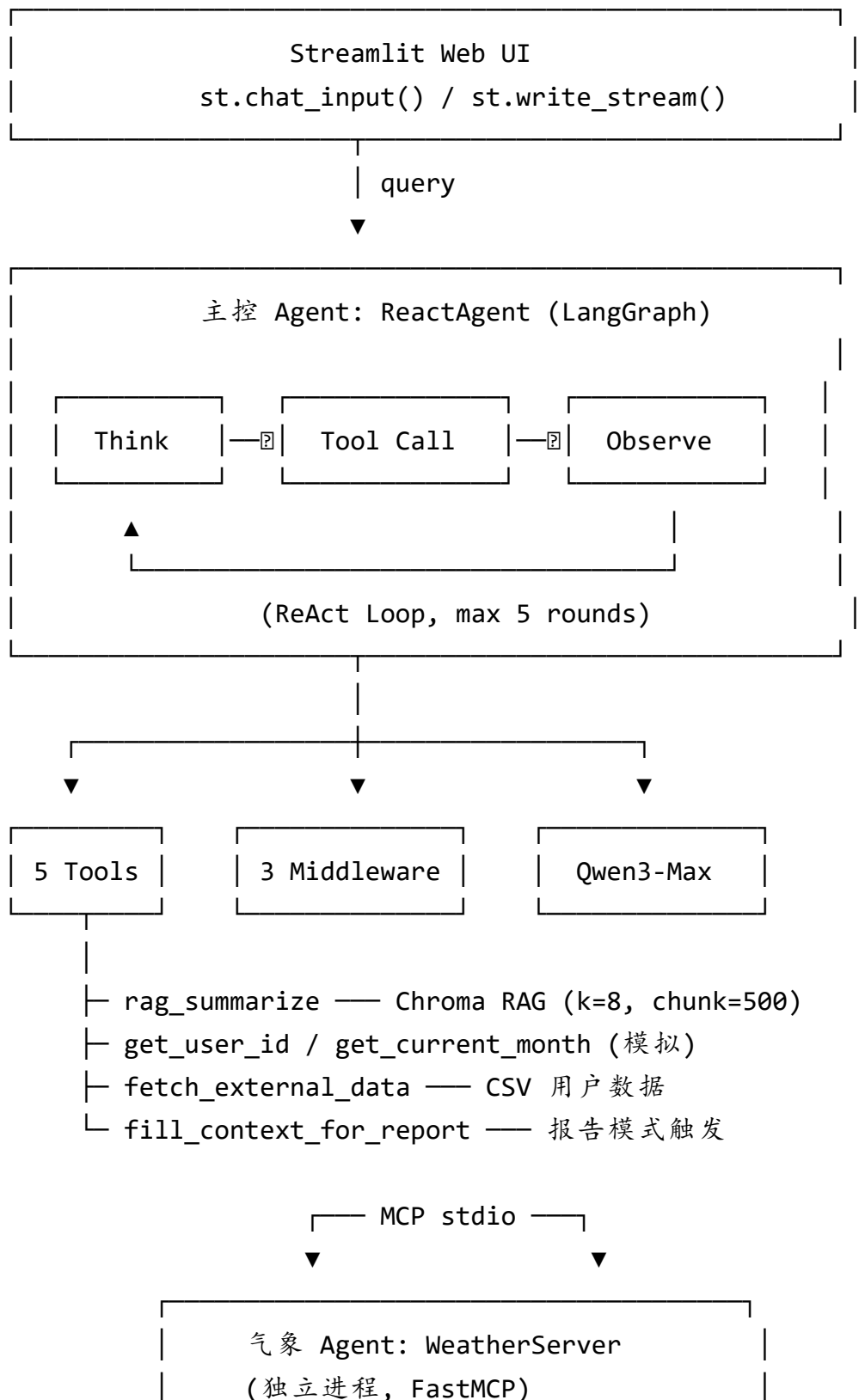
核心设计模式：抽象工厂（模型实例化）、中间件管道（AOP 横切）、MCP 包装器（异步转同步）。

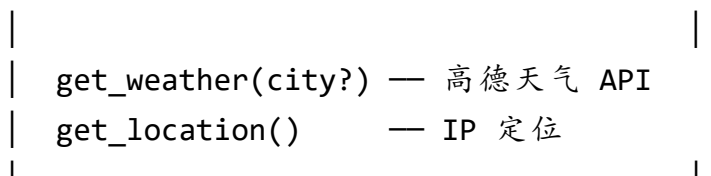
4.52.5 Test Spec 摘要

采用 RAGAS 四大指标评估 RAG 管道质量，5 组测试用例覆盖选购、故障、维护、宠物、对比等典型场景。

5 三、系统架构与设计

5.1 3.1 整体架构

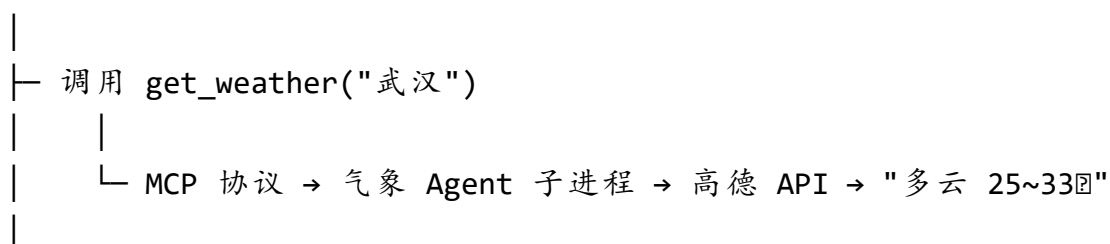




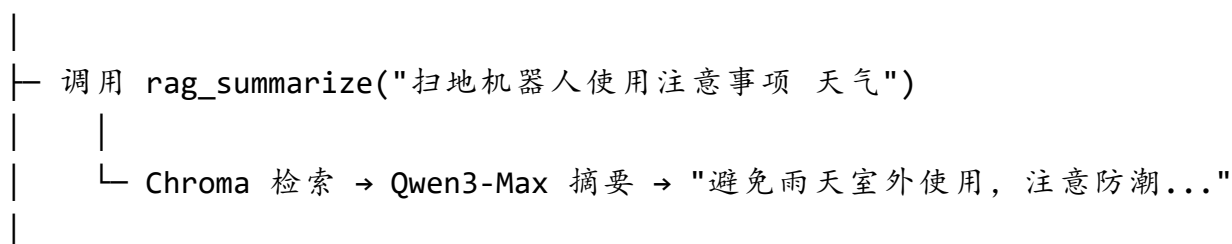
5.2 3.2 多智能体协作流程

用户: "武汉今天适不适合用扫地机器人?"

主控 Agent (Think) → 用户想知道武汉天气 + 对机器人的影响



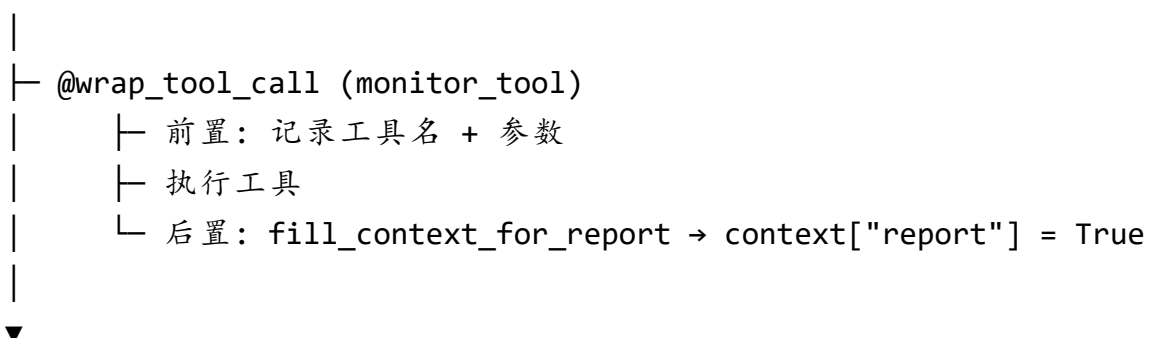
主控 Agent (Observe) → 天气良好, 可以正常工作



主控 Agent (最终回答) → "武汉今天多云, 25~33℃, 天气状况良好, 适合使用扫地机器人
建议在晴天模式下运行, 避免湿拖功能..."

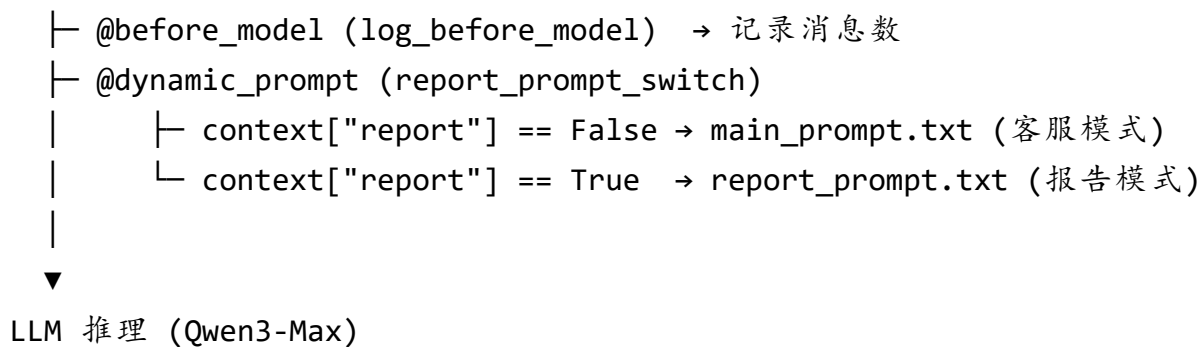
5.3 3.3 中间件管道

Tool Call 请求

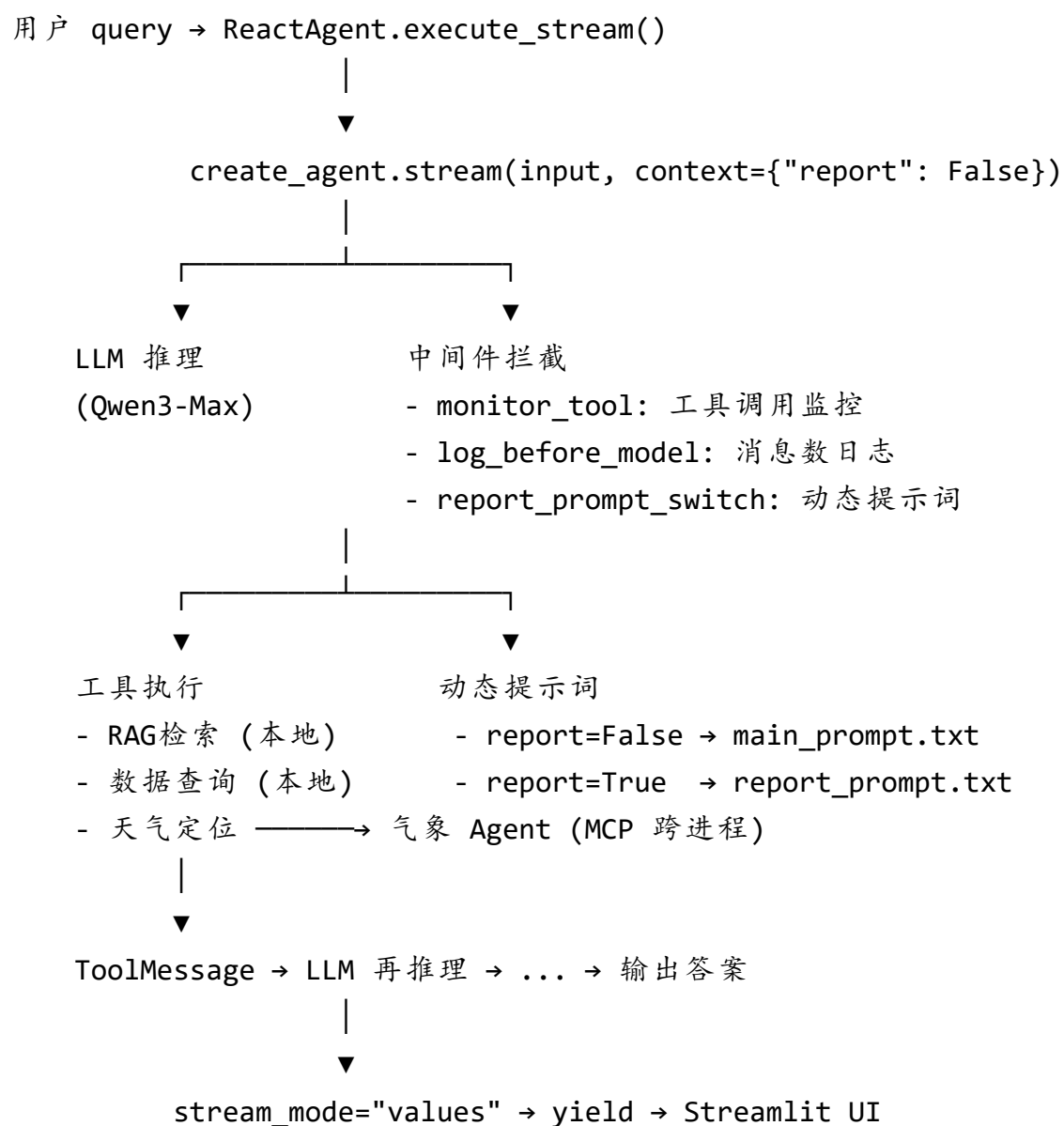


模型调用前

|



5.4 3.4 数据流



6 四、关键实现与代码展示

6.1 4.1 Agent 核心循环

```
# src/agent/react_agent.py
```

```
class ReactAgent:
    def __init__(self):
        mcp_tools = asyncio.run(self._init_mcp_tools())
        self.agent = create_agent(
            model=chat_model,                # Qwen3-Max
            system_prompt=load_system_prompt(),
            tools=[
                rag_summarize,
                *mcp_tools,                  # 气象 Agent 工具 (MCP)
                get_user_id,
                get_current_month,
                fetch_external_data,
                fill_context_for_report,
            ],
            middleware=[
                monitor_tool,                # @wrap_tool_call
                log_before_model,            # @before_model
                report_prompt_switch,        # @dynamic_prompt
            ],
        )

    def execute_stream(self, query: str):
        for chunk in self.agent.stream(
            {"messages": self.messages},
            stream_mode="values",
            context={"report": False},
        ):
            latest = chunk["messages"][-1]
            if latest.content:
                yield latest.content.strip() + "\n"
```

6.2 4.2 工具定义示例

```
# src/agent/tools/agent_tools.py
@tool(description=" 获取指定城市天气，不传城市则自动查询当前真实城市天气")
def get_weather(city: str = None) -> str:
    if not city:
        city = _resolve_current_city()
    adcode = city2adcode(city)
    resp = requests.get(WEATHER_URL, params={"city": adcode, "key": AMAP_KEY})
    data = resp.json()
    if data.get("status") != "1":
        return f" 天气查询失败: {data.get('info', '未知错误')}"
    forecast = data["forecasts"][0]["casts"][0]
    return f"{city}今天{forecast['dayweather']}, 气温{forecast['nighttemp']}~{for
```

6.3 4.3 MCP 气象 Agent

```
# src/mcp_server/weather_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("WeatherServer")

@mcp.tool()
def get_weather(city: str = None) -> str:
    """ 查询指定城市天气 """
    if not city:
        city = _resolve_city()
    adcode = _city2adcode(city)
    resp = requests.get(WEATHER_URL, params={"city": adcode, "key": AMAP_KEY})
    data = resp.json()
    forecast = data["forecasts"][0]["casts"][0]
    return f"{city}今天{forecast['dayweather']}, 气温{forecast['nighttemp']}~{for

@mcp.tool()
def get_location() -> str:
    """IP 定位获取当前城市"""
    return _resolve_city()
```

6.4 4.4 MCP 工具同步包装器

```
# src/agent/react_agent.py
@staticmethod
def _to_sync(async_tool):
    """MCP 异步工具 → LangGraph 同步 @tool"""
    @tool(description=async_tool.description or "")
    def wrapper(**kwargs):
        result = asyncio.run(async_tool.ainvoke(kwargs))
        if isinstance(result, list) and result:
            item = result[0]
            if isinstance(item, dict):
                return item.get("text", str(item))
            if hasattr(item, "text"):
                return item.text
        return str(result)
    wrapper.name = async_tool.name
    return wrapper
```

6.5 4.5 中间件实现

```
# src/agent/tools/middleware.py

@wrap_tool_call
def monitor_tool(request, handler):
    """ 工具调用监控 + 报告模式触发 """
    logger.info(f"[tool monitor] 执行工具: {request.tool_call['name']}")
    logger.info(f"[tool monitor] 传入参数: {request.tool_call['args']}")
    result = handler(request)
    if request.tool_call['name'] == "fill_context_for_report":
        request.runtime.context["report"] = True
    return result

@dynamic_prompt
def report_prompt_switch(request):
    """ 动态提示词切换 """
    if request.runtime.context.get("report"):
```

```
    return load_report_prompt()
return load_system_prompt()
```

6.6 4.6 配置系统（SDD 驱动）

```
# src/config/chroma.yml
collection_name: agent
persist_directory: chroma_db
k: 8                # 检索 Top-K
chunk_size: 500     # 分块大小
chunk_overlap: 50
data_path: src/data
```

7 五、测试与评估

7.1 5.1 测试策略

层级	方法	覆盖
单元测试	工具独立调用验证	7 个 Tool 逐个调通
集成测试	Streamlit 交互测试	天气查询 / RAG 问答 / 报告生成
评估测试	RAGAS 四大指标	5 组测试用例量化评估

7.2 5.2 RAGAS 评估结果

指标	分数	评级	说明
context_precision	0.82	良好	检索精准，极少无关文档
context_recall	0.87	良好	知识库覆盖充分，chunk_size=500 后大幅提升
faithfulness	0.72	良好	回答基于检索文档，幻觉风险低
answer_relevancy	0.76	良好	回答紧扣用户问题，无跑题

四项指标均 > 0.7，整体评级“良好”。

优化过程：- v1: k=3, chunk=200 → recall=0.27（较差），知识碎片化严重 - v2: k=8, chunk=500 → recall=0.87（良好），信息密度提升，召回率显著改善

7.3 5.3 功能测试

测试场景	输入	预期输出	结果
天气查询	“武汉今天天气怎么样”	返回天气 + 使用建议	
故障排查	“机器人无法回充怎么办”	检索文档 + 排查步骤	
选购咨询	“小户型适合买什么机器人”	检索选购指南 + 推荐	
维护保养	“扫地机器人日常怎么维护”	检索保养文档 + 建议	
报告生成	“帮我生成 6 月使用报告”	获取数据 + 切换模式 + 结构化报告	

8 六、系统升级与扩展

8.1 6.1 可扩展架构

当前架构天然支持以下扩展方向：

扩展点	方式	预期效果
新增知识库	放入 <code>src/data/</code> 运行 <code>vector_store</code>	自动加载，无需改代码
新增 MCP Agent	在 <code>_init_mcp_tools</code> 添加 <code>Server</code>	即插即用
切换 LLM	修改 <code>rag.yml</code> 配置	抽象工厂解耦，一行切换
新增工具	添加 <code>@tool</code> 函数 + 注册到 <code>tools</code> 列表	5 分钟集成
新增中间件	添加 <code>@wrap_tool_call</code> 等装饰器	AOP 无侵入

8.2 6.2 下一阶段计划

短期 (v1.1) - 接入真实用户数据（替换模拟 CSV）- 增加更多知识库文件（覆盖更多机型、品牌）- Docker 一键部署到云服务器（阿里云 ECS）

中期 (v2.0) - 引入 LangSmith Tracing 实现全链路可观测 - 增加第三个 Agent（如“数据分析 Agent”独立处理报告生成）- 接入 Ollama 本地模型，降低 API 成本

长期 (v3.0) - 多轮对话记忆持久化 (Redis / MemPalace 模式) - 前端升级为企业微信/钉钉 Bot 接入 - 知识库自动更新流水线 (CI/CD + 增量索引)

8.3 6.3 AI 能力演进路径

v1.0 (当前)		v2.0		v3.0
ReAct + RAG	→	Multi-Agent	→	Self-Learning
双智能体协作		3+ Agent 协作		知识库自更新
静态 Prompt		动态路由		意图自识别
离线评估		在线 Tracing		持续学习

9 七、课程总结

9.1 7.1 个人收获

1. 从“写代码”到“编排智能体”
传统开发关注 CRUD 和业务逻辑，而 Agent 开发的核心是定义工具、设计提示词、配置中间件管道。思维方式从“过程式”转向“声明式 + 编排式”。
2. SDD 方法论的价值
先写完整 Spec 再写代码，虽然前期投入大，但后续实现效率极高。API Spec 中定义的工具接口直接映射为 @tool 函数，规格即代码。
3. MCP 协议的工程价值
将天气服务拆为独立 Agent 进程，通过 MCP 协议通信。这种松耦合设计让服务可以独立开发、测试、部署，是微服务架构在 AI 领域的自然延伸。
4. RAG 评估的重要性
调参前 recall=0.27，调参后 0.87，数值驱动优化远胜于“凭感觉”。RAGAS 四项指标让 RAG 管道从黑盒变成可量化工程。

9.2 7.2 工程思维转变

- 以前：写一个函数，测试它 → 完成

- 现在：定义工具规格 → Agent 编排 → 中间件管道 → Benchmark 评估 → 迭代优化

Agent 开发是一个“设计 → 实现 → 评估 → 优化”的完整工程闭环，而非一次性编码。

9.3 7.3 对课程的建议

- 增加 MCP 协议实战课时，Agent 间通信是生产落地的关键
- 引入 RAGAS 评估教学，让“Agent 好不好用”有数据支撑
- 建议提供统一的云沙箱环境，降低环境配置门槛

9.4 附录：技术栈一览

类别	选型
LLM	阿里云 Qwen3-Max (ChatTongyi)
Embedding	DashScope text-embedding-v4
Agent 框架	LangGraph create_agent
向量数据库	Chroma (本地持久化)
协议	MCP (Model Context Protocol)
前端	Streamlit
外部 API	高德地图 (天气、IP 定位)
评估	RAGAS (context_precision / recall / faithfulness / answer_relevancy)
基础设施	Python 3.10, Docker, Git

9.5 附录：核心技术要素覆盖

要素	实现
SDD 规格驱动开发	docs/CS599_Specs.md 五章完整 SDD
工具使用 / Function Calling / MCP	7 Tools + MCP 气象 Agent
记忆机制	对话窗口 (AgentState) + Chroma 向量库 + MCP 外挂
状态管理与多步推理	LangGraph + ReAct (max 5 rounds) + Runtime Context
多智能体协作	主控 Agent + 气象 Agent (MCP stdio)
可观测性与评估	中间件日志 + RAGAS 四项指标